

API Client Integration Guide

This guide covers installing the `pagedown-client` package, configuring authentication, and adapting the bundled rate limiter for production use. It assumes familiarity with `pip`, environment variables, and basic HTTP concepts such as `GET / POST` and status codes like `429 Too Many Requests`.

Installation

Install the package from the internal index and confirm the `pagedown` command-line entry point is on `PATH`:

```
pip install pagedown-client
pagedown --version
export PAGEDOWN_API_KEY="sk_live..."
```

Configuration

The client reads `PAGEDOWN_API_KEY` from the environment by default, but a `config.yaml` file placed next to your project's entry point takes precedence if present. Set `timeout_seconds`, `base_url`, and `retries` to override the built-in defaults. A successful `client.ping()` call prints a short status line to stdout, for example:

```
client: connected
endpoint: https://api.example.com/v1
latency_ms: 42
```

A minimal `config.yaml` only needs to override what differs from the defaults:

```
base_url: https://api.example.com/v1
timeout_seconds: 30
retries: 3
```

Call `client.initialize()` once at process start; every subsequent `client.request(...)` call reuses the same underlying connection pool and respects the rate limiter described below.

Example: A Rate-Limited Request Queue

Every outbound call from `client.request()` first acquires a token from a `TokenBucketRateLimiter` instance. The reference implementation below is intentionally dependency-free — it is small enough to vendor directly into a codebase that cannot pull in the full SDK, and it is what `pagedown-client` itself wraps around every HTTP call. Read `acquire()` first: it is the one method application code actually calls.

```

"""Token-bucket rate limiter for outbound API requests.

Backs the pagedown-client package's built-in request queue.
Dependency-free by design, so it can be vendored into an
environment where the full SDK is unavailable.
"""

import time
import threading
from collections import deque
from dataclasses import dataclass
from typing import Callable, Deque, Optional

@dataclass
class RateLimiterConfig:
    """Configuration for one TokenBucketRateLimiter instance."""

    capacity: int = 60
    refill_amount: int = 60
    refill_interval_seconds: float = 60.0
    max_queue_depth: int = 500

class RateLimitExceeded(Exception):
    """Raised when a caller waits past max_queue_depth."""

class TokenBucketRateLimiter:
    """Thread-safe token-bucket limiter, bounded wait queue.

    Callers acquire a token before issuing a request. If no
    tokens are available, the caller blocks until either a
    token frees up or the queue depth limit is exceeded, in
    which case RateLimitExceeded is raised rather than

```

```

blocking indefinitely.
"""

def __init__(
    self, config: Optional[RateLimiterConfig] = None
) -> None:
    self._config = config or RateLimiterConfig()
    self._tokens = self._config.capacity
    self._lock = threading.Lock()
    self._condition = threading.Condition(self._lock)
    self._waiters: Deque[float] = deque()
    self._last_refill = time.monotonic()

def _refill_locked(self) -> None:
    now = time.monotonic()
    elapsed = now - self._last_refill
    interval = self._config.refill_interval_seconds
    if elapsed < interval:
        return
    steps = int(elapsed / interval)
    refill = steps * self._config.refill_amount
    capacity = self._config.capacity
    self._tokens = min(capacity, self._tokens + refill)
    self._last_refill = now

def acquire(self, timeout: Optional[float] = None) -> None:
    """Block until a token is free, or raise on overflow."""
    with self._condition:
        depth = len(self._waiters)
        limit = self._config.max_queue_depth
        if depth >= limit:
            raise RateLimitExceeded(
                f"queue depth {depth} exceeds "
                f"max_queue_depth={limit}"
            )

```

```

        self._waiters.append(time.monotonic())
    try:
        while True:
            self._refill_locked()
            if self._tokens > 0:
                self._tokens -= 1
                return
            ok = self._condition.wait(timeout=timeout)
            if not ok:
                raise RateLimitExceeded(
                    "timed out waiting for a token"
                )
    finally:
        self._waiters.popleft()

def release_hint(self) -> None:
    """Wake blocked callers to re-check token supply."""
    with self._condition:
        self._condition.notify_all()

def wrap(
    self, fn: Callable[..., object]
) -> Callable[..., object]:
    """Wrap fn so it acquires a token before calling."""

    def wrapped(*args: object, **kwargs: object) -> object:
        self.acquire()
        return fn(*args, **kwargs)

    return wrapped

def snapshot(self) -> dict:
    """Return a point-in-time view of limiter state."""
    with self._lock:
        return {

```

```

        "tokens": self._tokens,
        "capacity": self._config.capacity,
        "waiters": len(self._waiters),
        "last_refill": self._last_refill,
    }

def build_default_limiter() -> TokenBucketRateLimiter:
    """Build the limiter pagedown-client installs by default."""
    return TokenBucketRateLimiter(
        RateLimiterConfig(
            capacity=60,
            refill_amount=60,
            refill_interval_seconds=60.0,
            max_queue_depth=500,
        )
    )

```

The `wrap()` helper is the easiest integration point: pass it any callable and every invocation will transparently wait for a token first. Most applications only ever need `client.request()`, which already calls `acquire()` internally — reach for `wrap()` only when instrumenting a lower-level transport of your own.

Handling 429 Responses

Even with the client-side limiter enabled, a shared `PAGEDOWN_API_KEY` used by multiple processes can still exceed the server's own limit. When that happens, `client.request()` raises `RateLimitExceeded` with the server's `Retry-After` header attached as `error.retry_after_seconds`. Catch this specifically rather than a bare `except Exception`, since a truly failed request (a `5xx`, a network timeout) should usually be handled differently than a request that simply needs to wait and retry.

Summary

Install with `pip install pagedown-client`, set `PAGEDOWN_API_KEY`, and call `client.initialize()` once at startup. The bundled `TokenBucketRateLimiter` protects

every `client.request()` call automatically; vendor it directly (as shown above) if you need the same behavior in a tool that cannot depend on the full SDK.